PARALLEL AND DISTRIBUTED SYSTEM · TASK 2.3 SUBMISSION

Initial System Framework

Biliran Resort & Tour Booking System

Instructor	Engr. Eunel O. Peñaranda
Team	Member A — central node & integration · Member B — island node · Member C — mobile app · Member D — consoles, reports & testing
Date	30 October 2026 · Design v2 (includes adviser revisions)
Repository	github.com/Abylon10/BookingSystem

Problem in brief

Resorts and tour operators on Biliran's islands sell the same rooms, tour seats and boat trips to online travellers and to walk-in guests at the front desk, so the last unit can be sold twice; and island links to the mainland are unreliable, so a central-only system would stop the desk. The system gives every island its own node that owns its inventory, connects the nodes to a central node by gRPC, searches and reports in parallel, and keeps each island working offline.

Contents of this submission (as required by task 2.3)

Required item	Where
Framework document	§1 Objectives · §2 Scope and limitations · §3 Conceptual framework (IPO)
Architecture diagram	§4 System architecture (Figure 1)
Workflow	§5 Flowcharts: online booking, walk-in, arrival confirmation, mark unavailable, catalog change, registration and accounts
Module list	§4.1 Modules (M1–M11 with owners) · §4.2 Interfaces · §4.3 Data flow
Technology stack	§6 Technology and environment
Short review slides	§7 — 11 slides at the end of this file

Full design (data model, concurrency analysis, requirements, risks, plan): docs/System-Design-and-Initial-Considerations.pdf in the repository.

1. Objectives

1.1 General objective

To design and implement a parallel and distributed booking system for resort and tour packages in Biliran that supports online and face-to-face booking from a shared inventory without double booking, and continues to operate on an island when its connection to the central node is lost.

1.2 Specific objectives

1. Develop a mobile application (Expo / React Native) for customers to search packages, book, pay (simulated) and view their booking confirmation.
2. Develop an island node with a web console for receptionists (walk-in booking, arrival confirmation, marking services unavailable) and node admins (services, packages, prices, receptionist accounts).
3. Develop a central node with an admin panel for island registration, account management, and province-wide bookings and reports.
4. Guarantee consistency under concurrency: no inventory unit is sold twice, and each booking is checked in at most once.
5. Support partition tolerance: island operations, staff logins and local changes keep working offline and synchronise automatically on reconnection.
6. Apply and measure parallelism: parallel search fan-out across nodes and parallel report generation with worker threads, compared against sequential baselines.

2. Scope and limitations

2.1 Scope

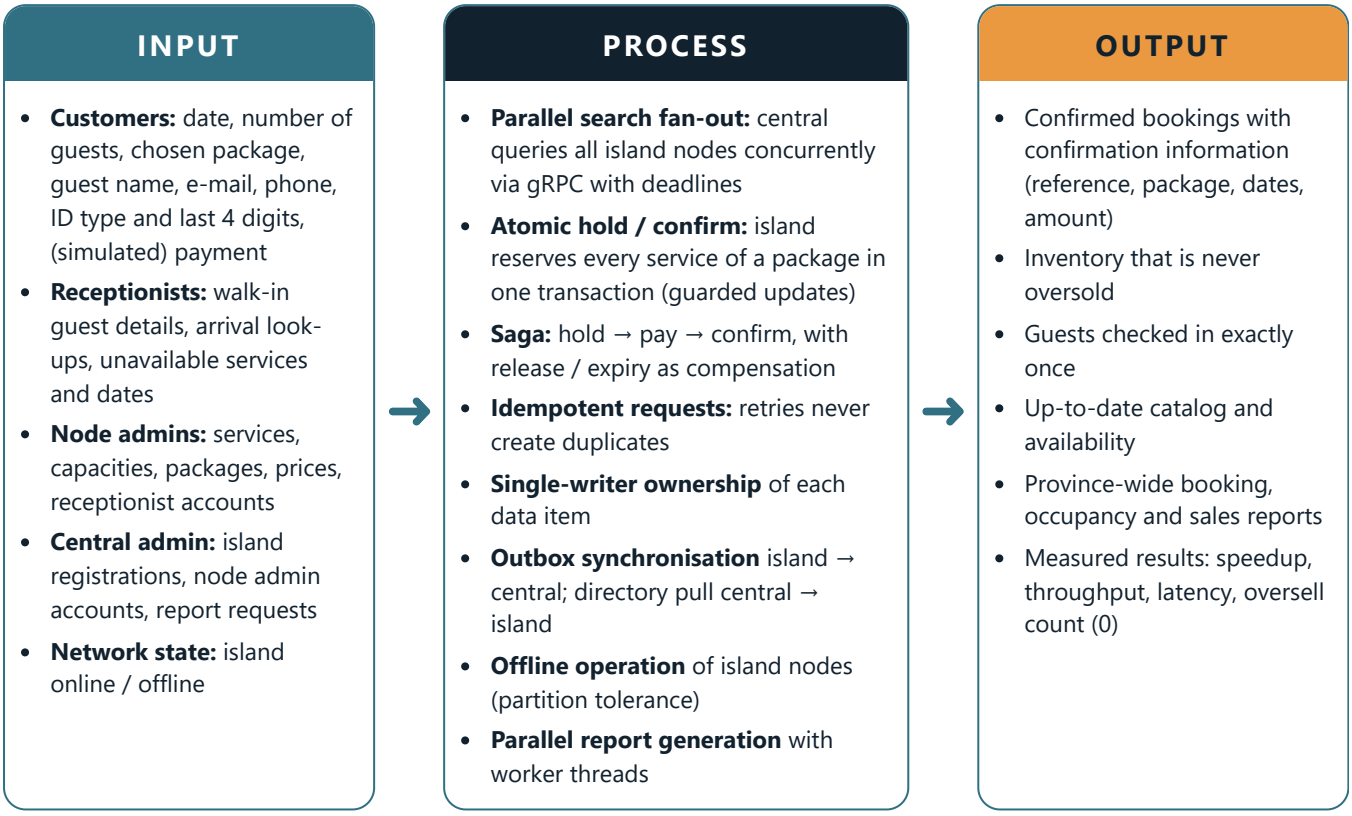
- Three island nodes (Biliran mainland, Maripipi, Higatangan) and one central node, demonstrated on four laptops connected by a phone hotspot.
- Package-based booking. Each package belongs to one island and is composed of services (room, tour, boat, meal).
- Online booking with a hold → pay → confirm flow; walk-in booking at the node console.
- Arrival confirmation by booking reference or name plus physical ID check (no tickets or QR codes).
- Manual blocking of unavailable services by the receptionist.
- Roles: central admin, node admin, receptionist, customer.
- Reports: bookings, occupancy and sales per island and package.

2.2 Limitations

- **Payment is simulated.** No real payment gateway (e.g. GCash, cards) is integrated.
- Packages cannot combine services from different islands.
- While an island is offline, its packages are **not sold online**.
- Customers have no accounts; bookings are looked up by reference and e-mail.
- Only the last four digits of a guest's ID are recorded; identity is verified visually by staff.
- Demonstrated on a local hotspot network; production hosting, TLS between nodes and backups are out of scope.
- Resorts and tour operators are fictional; prices and capacities are sample data.

3. Conceptual framework

The study follows the Input–Process–Output (IPO) model. Inputs from the four user groups are processed by the distributed nodes using parallel and concurrency-safe techniques, producing consistent bookings, confirmations and reports. Evaluation results feed back into the design.



3.1 Key concepts applied

Concept	How it appears in the system
Distribution / data partitioning	Data partitioned by island; each node stores and owns its own island's data.
Remote procedure call	gRPC contract (`proto/booking.proto`) between central and island nodes.
Parallelism (task)	Concurrent RPCs to all islands during search; total time ≈ slowest island instead of the sum.
Parallelism (data)	Report data split across worker threads (1 / 2 / 4 / 8) and merged.
Concurrency control	Atomic conditional updates and transactions; single writer per data item.
Fault tolerance / partition	Deadlines, offline mode, outbox with retries, de-duplication by event id.
Consistency trade-off (CAP)	Consistency chosen over availability for online sales of an unreachable island.

4. System architecture

Four laptops and the customers' phones share one phone hotspot. Laptop 1 is the central node; Laptops 2–4 each run one island node. The same island software runs on every island laptop with a different island id.

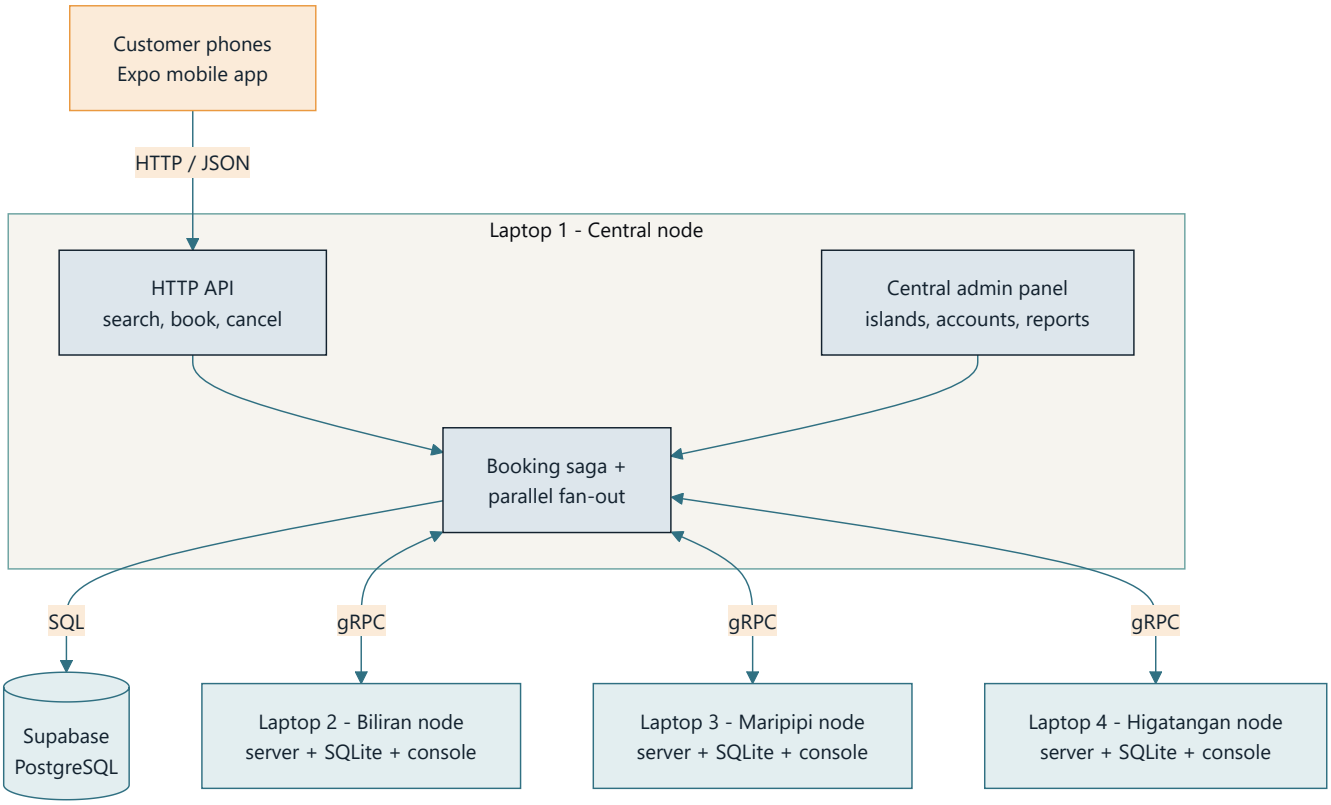


Figure 1. Distributed architecture and communication paths

Component	Runs on	Responsibilities
Mobile app	Customer phones (Expo Go)	Search packages, book, simulated payment, confirmation info, my bookings, cancel
Central node	Laptop 1	HTTP API for the app, parallel fan-out, booking saga, central admin panel, island registration, admin accounts, copies of all bookings for reports
Supabase (PostgreSQL)	Cloud, used by central only	Central storage: islands, accounts, catalog copy, booking copy, sync events
Island node	Laptops 2–4	Services, packages, inventory, bookings (single owner), receptionist accounts, outbox, node console
Node console	Browser on each island laptop	Login; receptionist pages; node admin pages

4.1 Modules

ID	Module	Runs on	Responsibility	Owner
M1	Customer mobile app	Phones	Search packages, book, simulated payment, confirmation info, my bookings, cancel	Member C
M2	Central API	Central	HTTP endpoints for the app; input validation; idempotency keys	Member A
M3	Booking orchestrator	Central	Parallel fan-out search; hold → pay → confirm saga; release on failure	Member A
M4	Central admin panel	Central	Island registration, node admin accounts, all bookings, reports	Member A
M5	Central sync service	Central	Receives island events (`PushEvents`); serves directory changes (`PullDirectory`)	Member A
M6	Island booking engine	Each island	Inventory, packages, hold/confirm/release, walk-in booking, arrival confirmation, blocking	Member B
M7	Island catalog & accounts	Each island	Services, packages, prices; receptionist accounts; local login (scrypt)	Member B
M8	Island sync agent	Each island	Outbox sender with retries; directory puller; partition switch	Member B
M9	Node console	Island laptops	Receptionist pages and node admin pages (web)	Member D
M10	Report workers	Central	Occupancy and sales reports computed in parallel with worker threads	Member D
M11	Test and benchmark tools	Any laptop	Race tests, load tests, sequential vs parallel benchmarks	Member D

4.2 Interfaces

Interface	Protocol	Operations
App → Central (M1→M2)	HTTP / JSON	`GET /api/packages?day&guests` · `POST /api/bookings` (hold) · `POST /api/bookings/:id/pay` · `GET /api/bookings/:ref?email` · `POST /api/bookings/:id/cancel`
Central → Island (M3→M6)	gRPC `IslandService`	`GetCatalog` · `SearchAvailability` · `HoldBooking` · `ConfirmBooking` · `ReleaseHold` · `CancelBooking` · `GetBooking` · `Ping`
Island → Central (M8→M5)	gRPC `CentralService`	`PushEvents` (bookings, catalog changes, receptionist accounts) · `PullDirectory` (registration, admin accounts)
Console → Island (M9→M6/M7)	HTTP / JSON	`/api/login` · `/api/walkin` · `/api/arrivals` (search, confirm) · `/api/blocks` · `/api/services` · `/api/packages` · `/api/staff` · `/api/partition`
Admin panel → Central (M4)	HTTP / JSON	`/admin/api/islands` · `/admin/api/accounts` · `/admin/api/bookings` · `/admin/api/reports`
Central → Supabase	PostgreSQL (supabase-js)	Islands, accounts, catalog copy, booking copy, events

4.3 Data flow

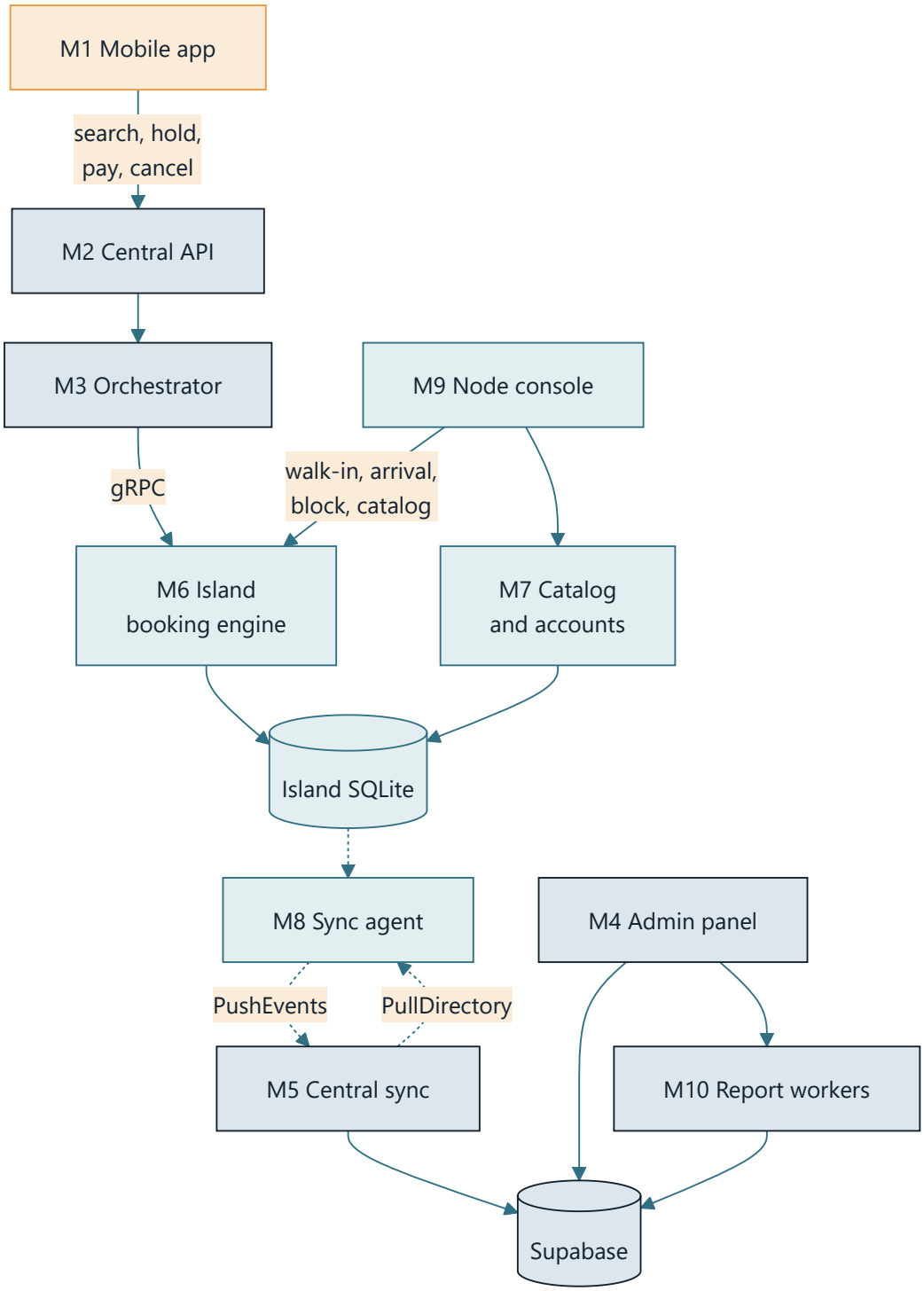


Figure 2. Main data flows. Solid arrows: requests in the booking path; dashed: background synchronisation.

1. **Booking path:** app → central API → orchestrator → owning island's booking engine → island database. The answer returns the same way.
2. **Walk-in and arrival path:** node console → island booking engine → island database (no central involvement).
3. **Island → central:** every local change also writes an outbox event; the sync agent pushes it to central, which stores copies in Supabase.
4. **Central → island:** registration and admin accounts flow down when the island pulls the directory.
5. **Reports:** admin panel → report workers read the booking copies in parallel.

5. Workflow (flowcharts)

5.1 Online booking (customer → central → island)

Part 1 — Parallel search

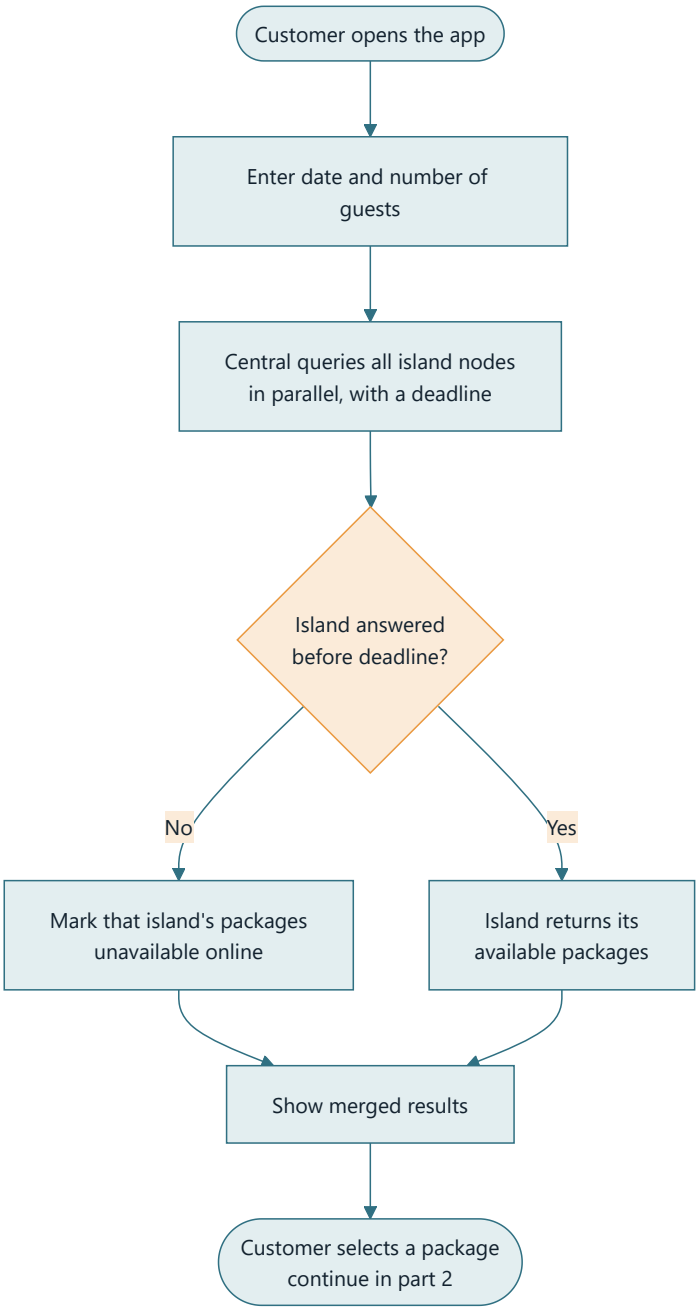


Figure 3. Central asks every island at the same time; an island that misses the deadline is shown as unavailable

Part 2 — Hold, pay, confirm (saga)

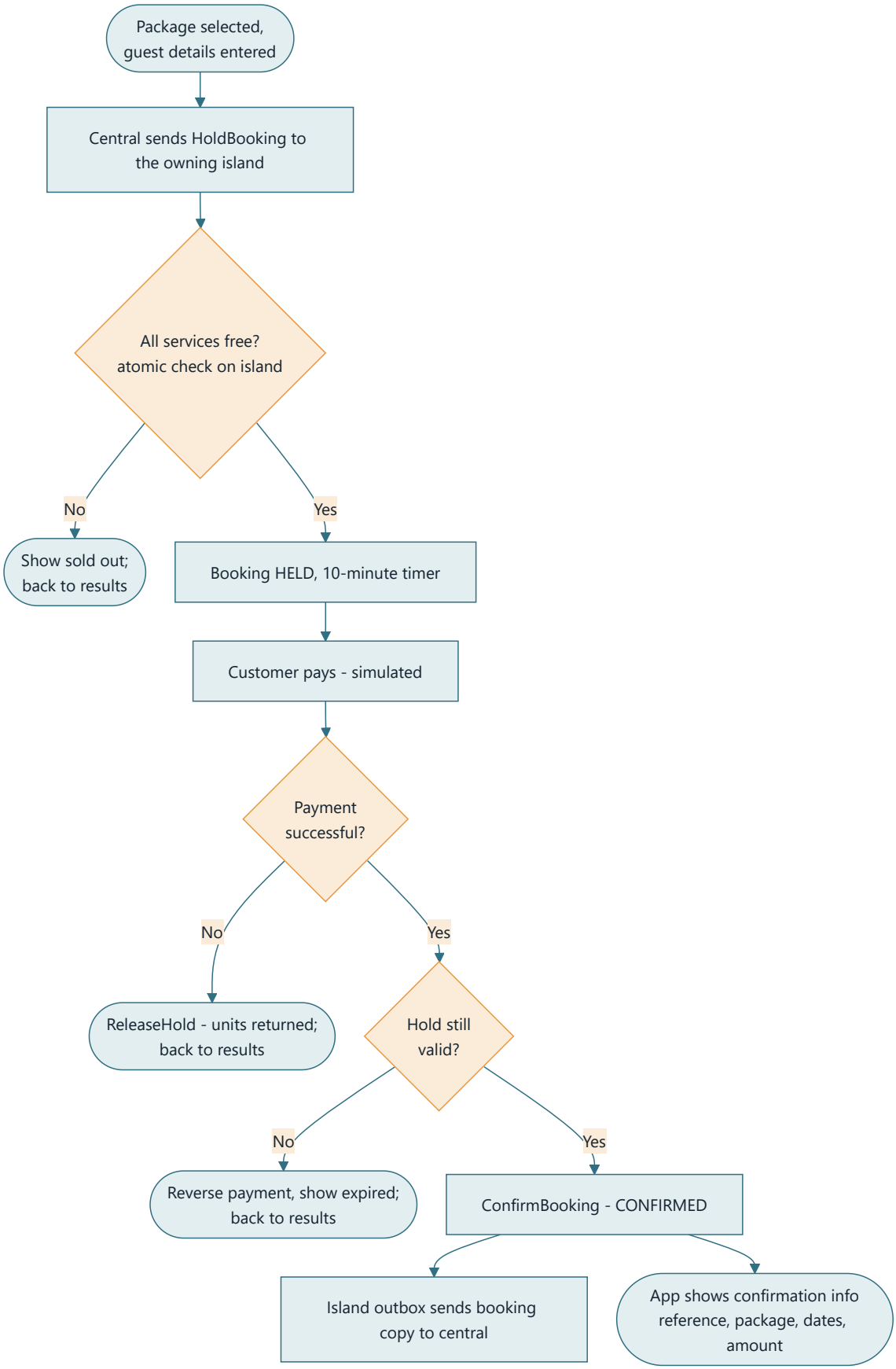


Figure 4. The owning island reserves all services atomically; release or expiry undoes the hold

5.2 Walk-in booking (receptionist)

> Online and walk-in bookings both go through the island node that owns the inventory, so the race for the last unit is decided by one atomic update on one database.

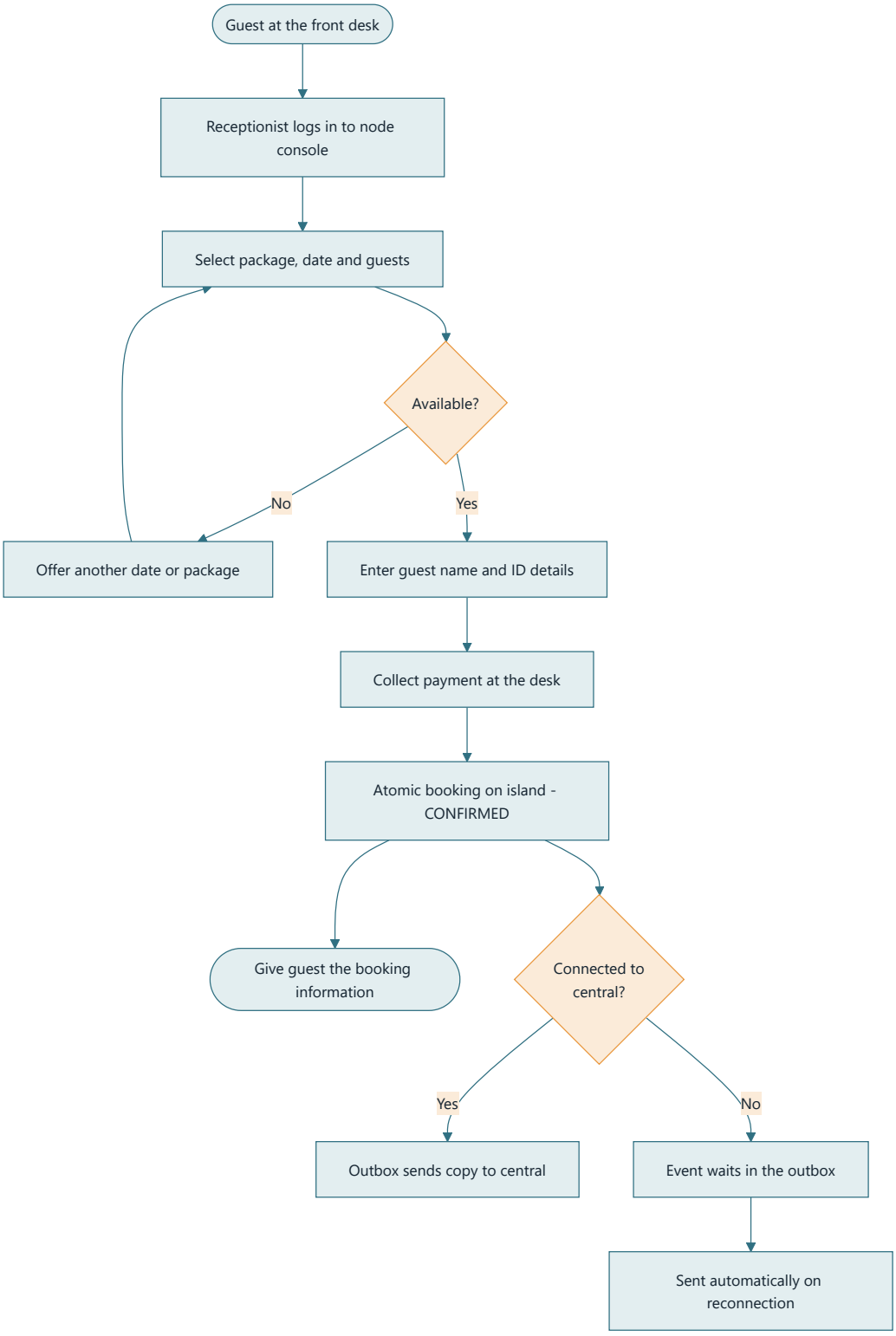


Figure 5. Walk-in booking from the same inventory as online booking

5.3 Arrival confirmation (no tickets)

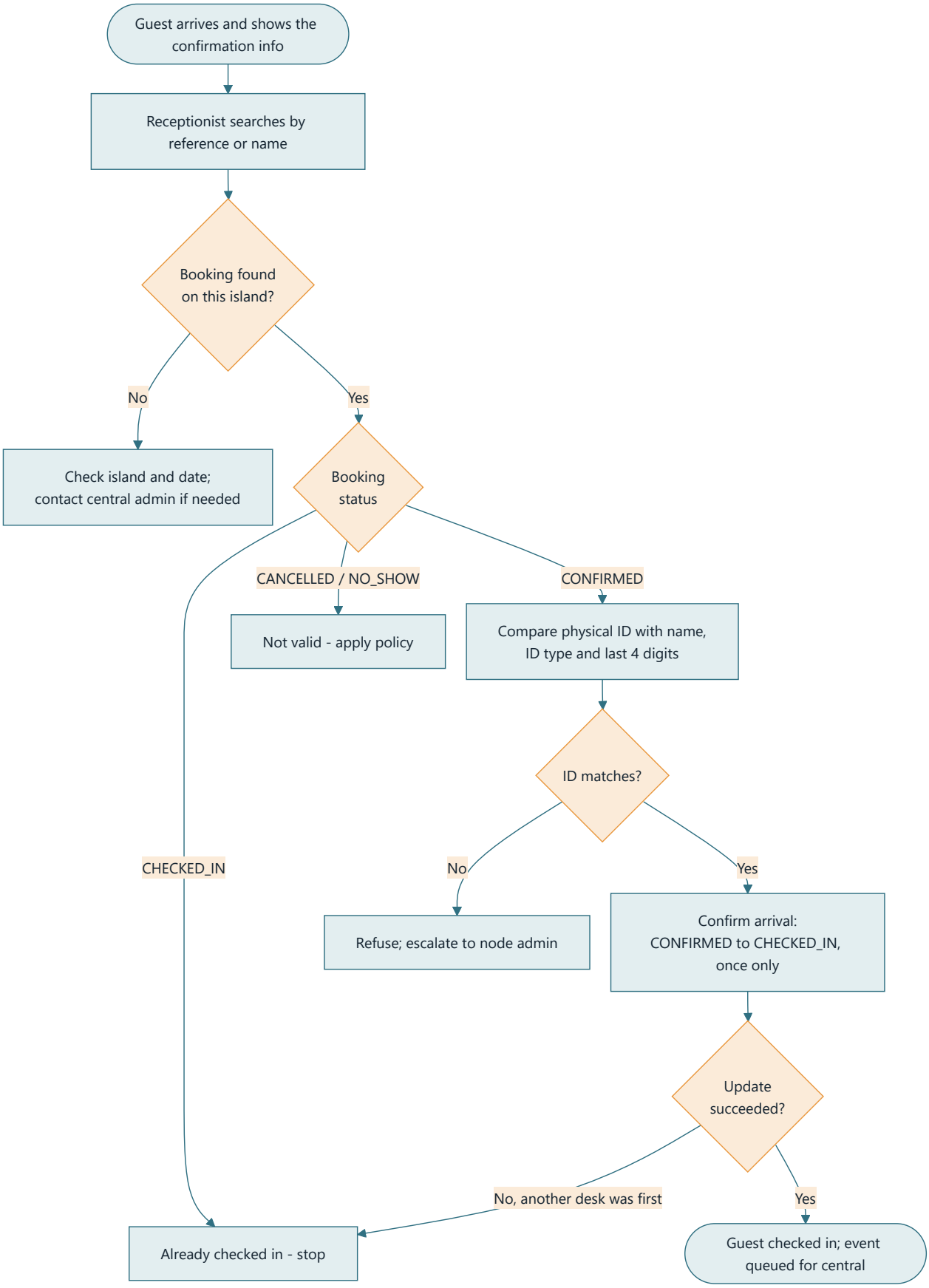


Figure 6. The guest shows the confirmation information received after payment; staff verify the ID

5.4 Mark service unavailable (receptionist)

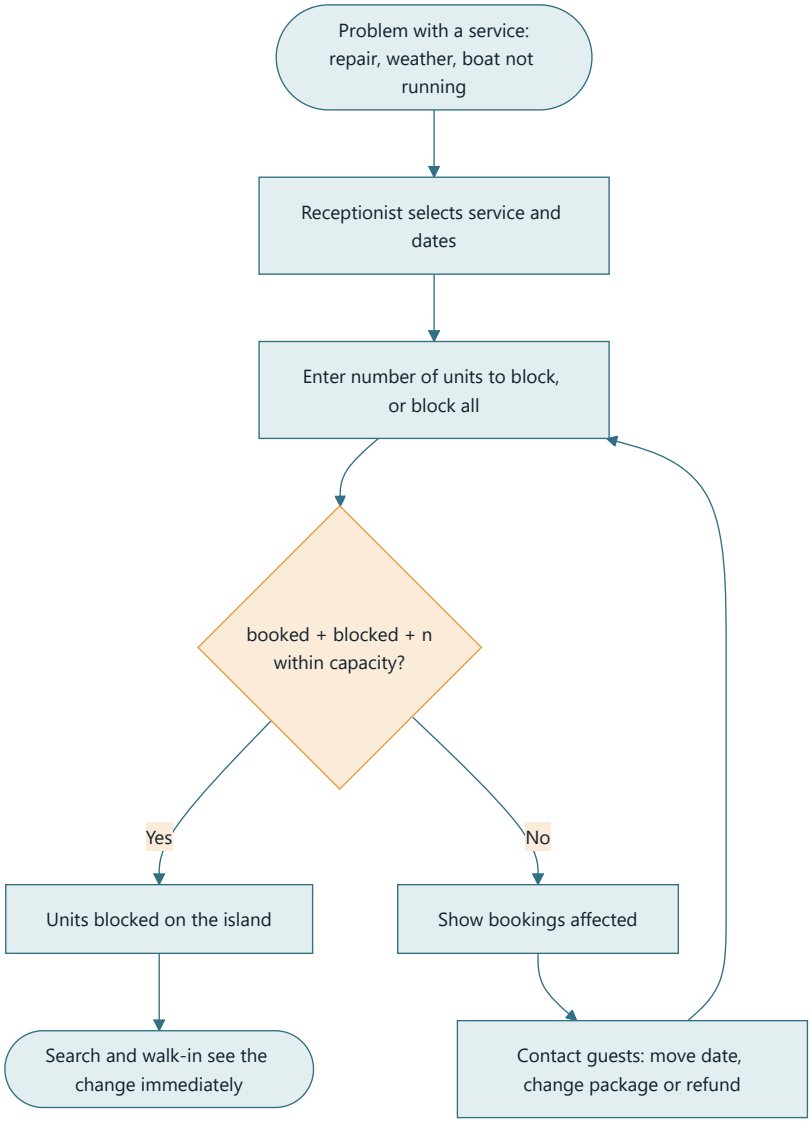


Figure 7. Manual input of unavailable services (repairs, weather, boat not running)

5.5 Catalog change by node admin

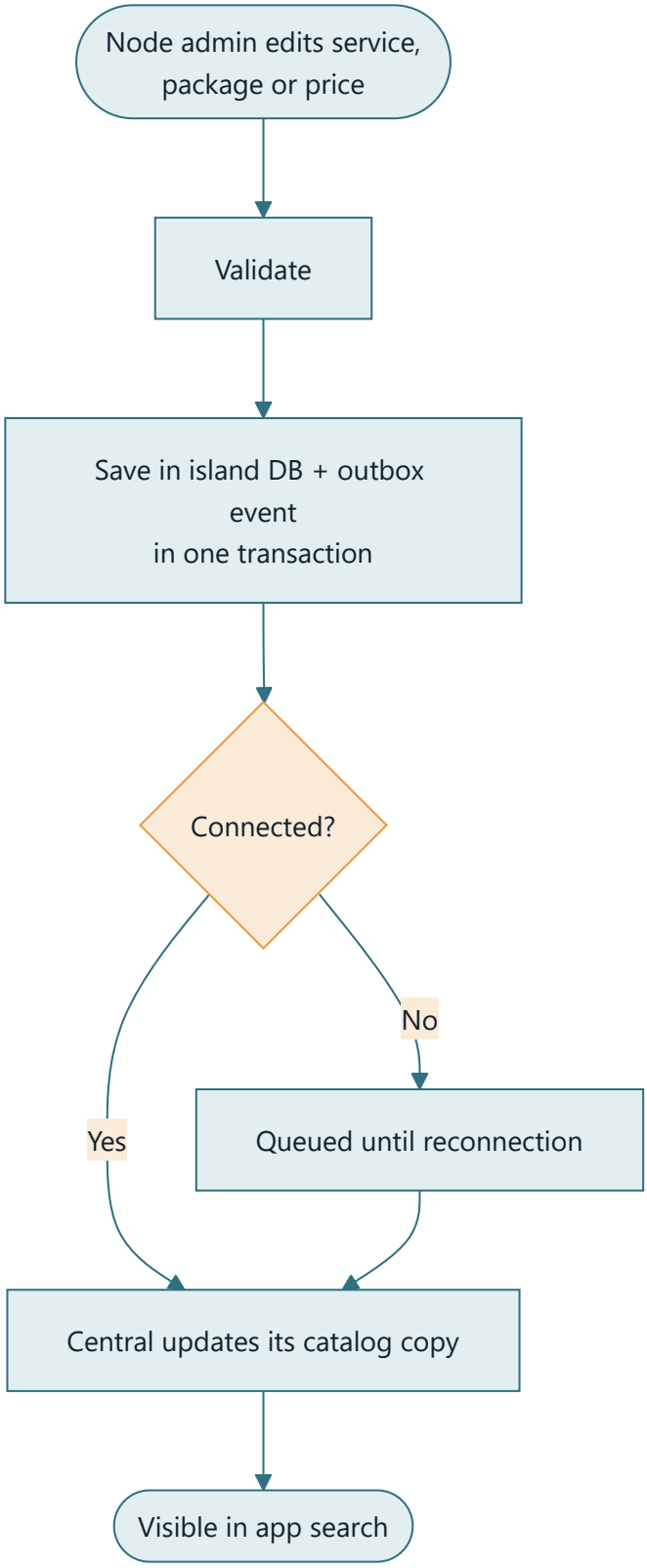


Figure 8. Changes are saved locally first (works offline), then synchronised to central

5.6 Island registration and accounts

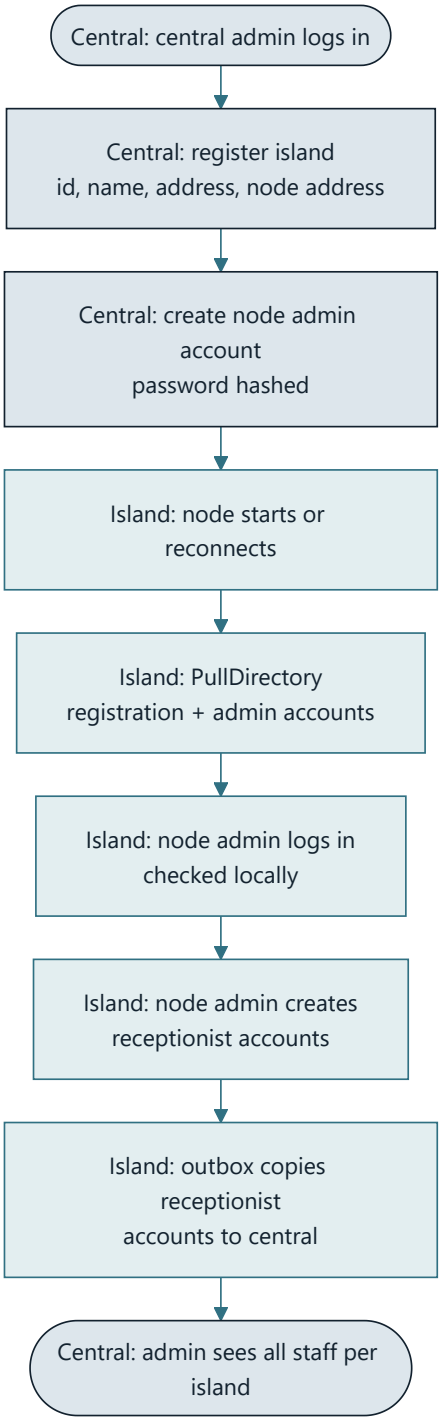


Figure 9. Central admin registers islands and node admins; node admins create receptionists (dark = central, teal = island)

Login

Passwords are stored only as salted scrypt hashes. Each node verifies logins against its local copy of the accounts, so staff can log in while the island is offline. Sessions expire after inactivity; disabled accounts stop working at the next directory pull.

6. Technology and environment

6.1 Software stack

Layer	Technology	Reason
Mobile app	Expo SDK 58 (React Native, TypeScript)	One code base for Android and iPhone; runs in Expo Go without app-store builds
Central and island servers	Node.js 24+ with TypeScript, Express	Same language as the app; runs TypeScript directly
Node-to-node communication	gRPC (`@grpc/grpc-js` , Protocol Buffers)	Typed RPC contract, deadlines, efficient binary protocol
Island storage	SQLite (built-in `node:sqlite`)	Embedded, works offline, transactional
Central storage	Supabase (PostgreSQL)	Hosted database with dashboard; row-level security
Consoles / admin panel	HTML, CSS, JavaScript served by each node	No installation on staff laptops
Parallel reports	`node:worker_threads`	True parallel CPU work in Node.js
Passwords	`node:crypto` scrypt	Memory-hard password hashing, no extra dependency
Version control	Git + GitHub	Team collaboration, history

6.2 Hardware and network

Item	Specification
Laptops (4)	Windows, Node.js 24+, modern browser; one central, three island nodes
Phones (4)	Android or iPhone with Expo Go supporting SDK 58
Network	One phone hotspot (all 8 devices); mobile data needed only for Supabase
Ports	Central HTTP 4000 / gRPC 50050; islands HTTP 8081–8083 / gRPC 50051–50053

6.3 Repository layout

Folder	Contents
`app/`	Expo mobile app
`central/`	Central server, admin panel, Supabase migrations
`island/`	Island node server, SQLite, node console
`proto/`	gRPC contract
`shared/`	Configuration and RPC helpers used by all servers
`loadtest/`	Race, search and load experiments
`config/`	Network addresses and seed data

SECTION 7

Short review slides

11 slides follow (16:9 pages)

PARALLEL AND DISTRIBUTED SYSTEM · INITIAL FRAMEWORK REVIEW

Biliran Resort & Tour Booking System

Online and face-to-face booking of island packages — one inventory, no double booking, works offline

Member A · Member B · Member C · Member D

Instructor: Engr. Eunel O. Peñaranda · 30 October 2026

Problem and objectives

Biliran's resorts and tours sell to online travellers and walk-in guests at the same time

Problem

- Online and walk-in sales draw from the **same rooms, seats and boats** — the last unit can be sold twice
- Island links to the mainland are **unreliable**; a central-only system stops the front desk
- Each resort manages its own services, but registration, accounts and reports need one place
- Searching islands one after another gets slower with every island

Objectives

- Book packages **online (app)** and **face-to-face (desk)** from one inventory
- **Zero** oversold units and double check-ins under concurrency
- Islands keep operating **offline** and sync automatically
- **Measure** parallel speedup: search fan-out and reports

Scope and limitations

Prototype demonstrated on 4 laptops and 4 phones over one hotspot

In scope

- 3 island nodes (Biliran mainland, Maripipi, Higatangan) + 1 central node
- Packages per island (room + tour + boat + meal)
- Online hold → pay → confirm; walk-in booking at the desk
- Arrival confirmation: booking info + physical ID check
- Receptionist marks services unavailable
- Central admin, node admins, receptionists

Limitations

- Payment is **simulated**
- Packages cannot span islands
- Offline island → its packages not sold online
- No customer accounts (reference + e-mail)
- Only last 4 ID digits stored (RA 10173)
- Fictional businesses, sample prices

Parallel and distributed components

Where the course concepts live in the system

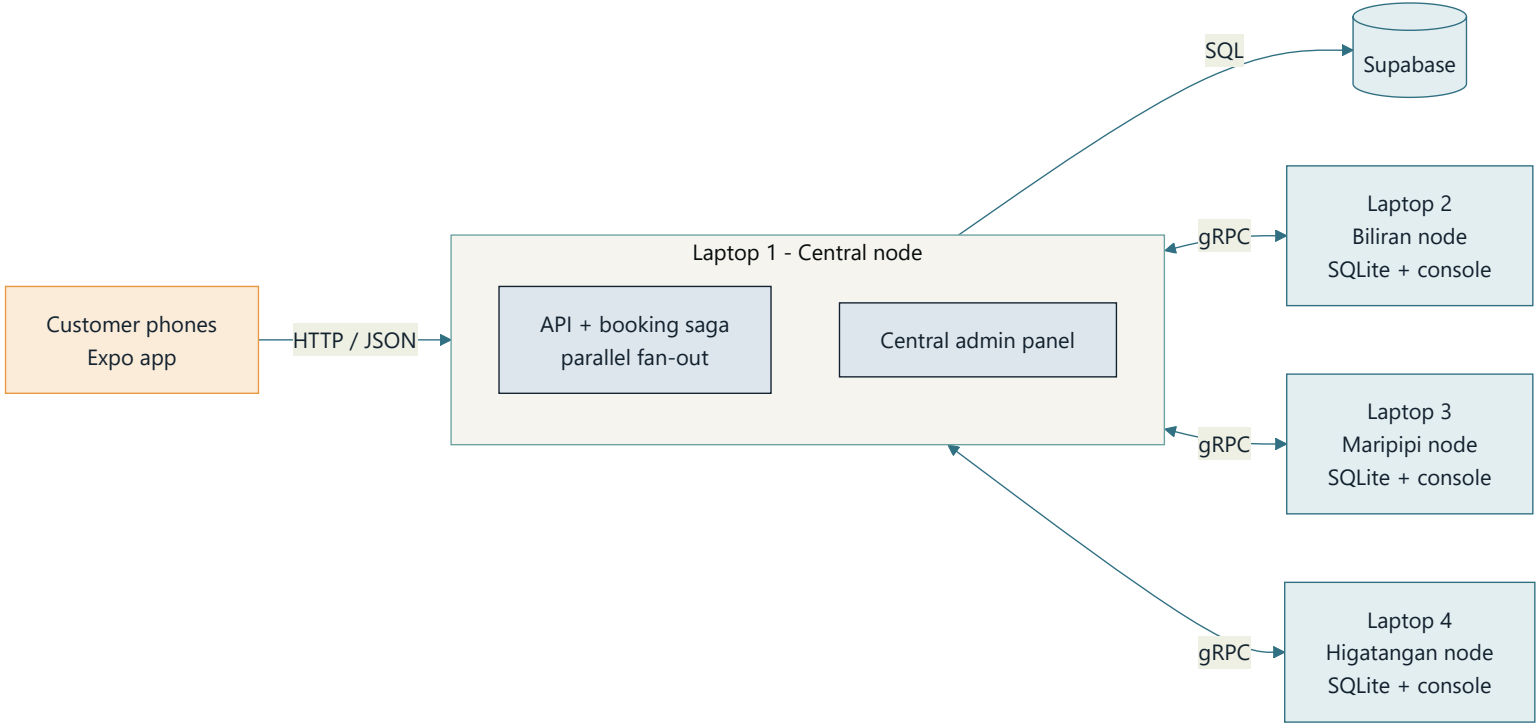
Distributed

- Data **partitioned by island**: each node owns its services, inventory and bookings
- Nodes communicate by **remote procedure calls (gRPC)** with deadlines
- **Outbox sync** with retries and de-duplication
- **Partition tolerance**: islands work offline, sync on reconnect
- One writer per data item — no conflicting edits

Parallel / concurrent

- **Parallel fan-out**: central queries all islands at once
- **Concurrent bookings** made safe with atomic guarded updates and transactions
- **Worker threads** compute occupancy and sales reports (1/2/4/8)
- **Load generators** fire N simultaneous requests to test and measure

System architecture



Conceptual framework (IPO)

INPUT

- Customer: date, guests, package, guest details, payment
- Receptionist: walk-ins, arrivals, unavailable services
- Node admin: services, packages, prices, receptionists
- Central admin: islands, node admins, report requests
- Network state: island online / offline

→

PROCESS

- Parallel search fan-out over gRPC
- Atomic hold / confirm on the owning island
- Saga: hold → pay → confirm, release on failure
- Idempotent requests; single writer per item
- Outbox sync and directory pull
- Offline operation of island nodes
- Parallel reports with worker threads

→

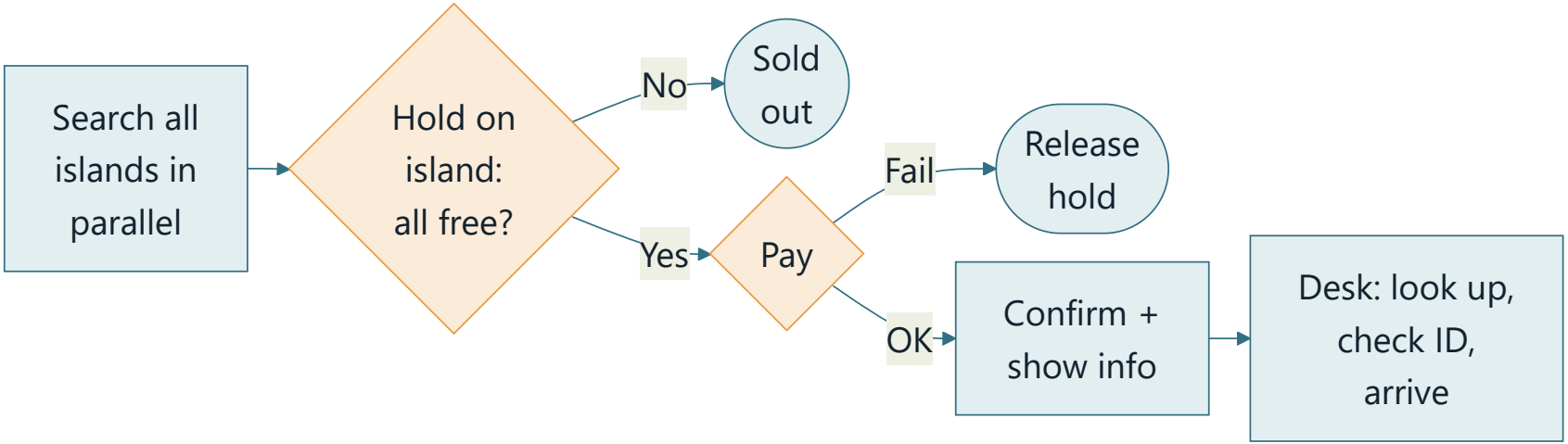
OUTPUT

- Confirmed bookings + confirmation info
- Inventory never oversold
- Guests checked in exactly once
- Province-wide reports
- Measured speedup, throughput, latency

Feedback: race, load, partition and speedup tests refine the design.

Main workflow: online booking

Walk-ins use the same island inventory directly from the node console



Consistency and offline behaviour

Race	Solution
Online vs walk-in, last room	Both go to the owning island; one atomic guarded update decides
Block a room while it is booked	Same rule: booked + blocked ≤ capacity
Retry after timeout	Idempotency key returns the same booking
Two desks confirm one arrival	CONFIRMED → CHECKED_IN succeeds once
Sync event delivered twice	De-duplicated by event id

Island offline

- Walk-ins, arrivals, logins keep working
- Changes queue in the outbox
- Its packages show **unavailable online** — never sell what the island cannot confirm (consistency over availability)
- On reconnect: outbox drains, directory pulled

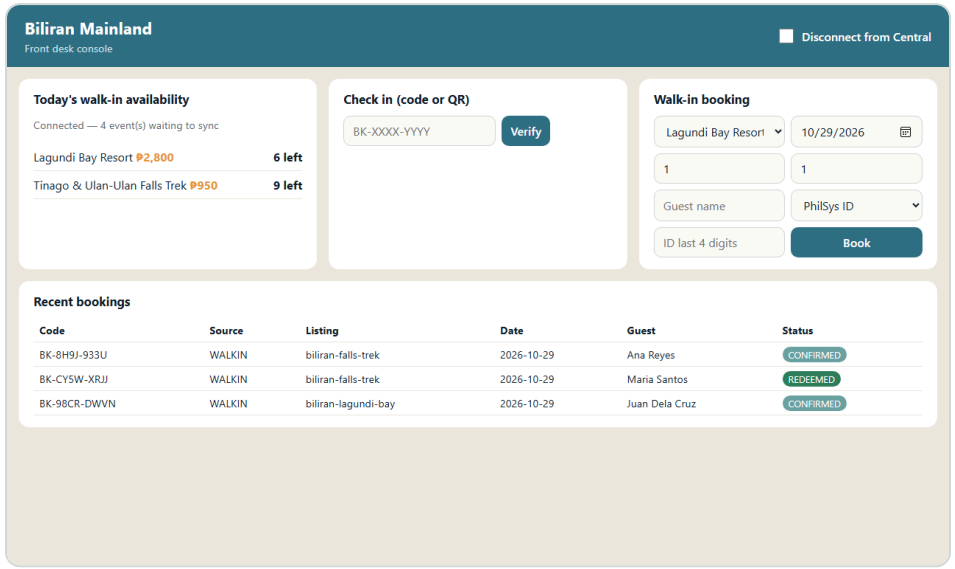
Modules and technology stack

Module	Owner
M1 Customer mobile app	Member C
M2–M5 Central API, orchestrator, admin panel, sync	Member A
M6–M8 Island booking engine, catalog & accounts, sync agent	Member B
M9 Node console (receptionist, node admin)	Member D
M10 Report workers · M11 Test & benchmark tools	Member D

Layer	Technology
App	Expo (React Native, TypeScript)
Servers	Node.js + TypeScript, Express
Node-to-node	gRPC (Protocol Buffers)
Island storage	SQLite (`node:sqlite`)
Central storage	Supabase PostgreSQL
Parallel reports	`worker_threads`

Status and early evidence

Baseline prototype (v1) running; reworking to the adviser-approved v2 design



2.93×
parallel vs sequential search across 3 islands (150 ms simulated link, 10 runs)

0 oversold
30 concurrent bookings for 6 rooms → exactly 6 booked

1 of 2
parallel double check-in → exactly one succeeds

Preliminary, measured on one laptop. Final measurements on the 4-laptop hotspot with 100 / 500 / 1,000 requests.

Next steps and items for approval

Plan to 20 November

- **Oct 30 – Nov 3:** v2 baseline, hotspot test with all 8 devices
- **Nov 2 – 10:** core features, gRPC v2, offline sync
- **Nov 9 – 13:** integration, tests, performance evidence
- **Nov 13 – 18:** documentation, slides, mock defense, fixes
- **Nov 19 – 20:** final defense and live demo

For approval

- Framework and architecture as presented
- Simulated payment (no real gateway)
- Offline island = unavailable online
- Package price per package (up to N guests)
- Cancellation: full refund up to 48 h before